Домашнее задание 10 — CompletableFuture

- **Дедлайн:** 21.10.2026
- **Тема:** CompletableFuture (асинхронное программирование)
- **Форма проверки:** задание с самопроверкой
- **Вариант итогового ДЗ:** 3 — парсер вакансий
- **Модуль:** `hw10.completable` в Maven-проекте `app/`

---

1. Что требовалось и что сделано

Текст задания:

1. По списку идентификаторов получить данные из внешнего источника (API, БД, файл, парсер). Реализовать метод, который асинхронно через `CompletableFuture` получает данные по каждому идентификатору, применяет преобразование, **объединяет два независимых асинхронных результата в один объект** и возвращает список итоговых объектов, собранных параллельно. Использовать `thenCombine`, `thenApply`, `thenAccept`. Метод: `public List<FinalResult> fetchAllDataAsync(List<String> ids)`.
2. Обработка ошибок через `handle` / `exceptionally` (сбой одной задачи не ломает операцию), собственный `Executor` вместо `ForkJoinPool.commonPool`, пул с именованными демон-потоками, `thenApplyAsync(…, executor)` для тяжёлых преобразований, корректное завершение `ExecutorService` при остановке приложения, логирование с именем потока и ID запроса, имитация ошибки в одном источнике (частичный результат), сравнение времени асинхронной и синхронной версий.
3. Отчёт: описание этапов, промежуточные результаты, выводы.

Сделано:

Что	Где
Локальный API-источник: данные вакансии и справочник компаний	`server.VacancyStubServer`
HTTP-клиент двух источников, разбор JSON	`source.HttpVacancyClient`, `source.FlatJson`
Два независимых источника-интерфейса	`source.VacancyDetailsSource`, `source.CompanyInfoSource`
Итоговый объект и сбой	`model.FinalResult`, `model.FetchFailure`
Конвейер: `supplyAsync` → `thenCombine` → `thenApplyAsync` → `handle` → `thenAccept`	`service.AsyncVacancyService`
Тяжёлое преобразование (нормализация + оценка)	`service.VacancyScoring`
Синхронная версия для сравнения	`service.AsyncVacancyService#runSync`
Итог прогона (время, потоки, сбои)	`service.FetchOutcome`
Строки отчёта	`report.AsyncFetchReport`

Что	Где
Демонстрация	`CompletableFutureDemo`, запуск `--app.hw=hw10`
Настройки `app.hw10.*`	`config.Hw10Properties`, `config.Hw10Configuration`

Предметная область — парсер вакансий: идентификатор — номер вакансии (`v-01…v-12`), первый источник отдаёт заголовок, город и зарплатную вилку, второй — компанию, её рейтинг и число открытых вакансий.

2. Этапы выполнения

Этап 1. Асинхронная выборка и объединение двух источников

Для каждого идентификатора запускаются **две** задачи: `supplyAsync(details)` + `supplyAsync(company)`. Обе идут на собственном пуле `hw10-async-1…6`. Результаты объединяются в `FinalResult` через `thenCombine` (два независимых результата → один объект), затем объект проходит тяжёлое преобразование в `thenApplyAsync(…, executor)` и попадает в приёмник через `thenAccept`. Сборка всех конвейеров — `CompletableFuture.allOf(...).join()`; порядок итогового списка — порядок входных идентификаторов, а не порядок завершения задач.

Фактический вывод этапа (12 идентификаторов, пул 6, задержка источника 120 мс на запрос, преобразование 10 мс):

```
Этап 1. Асинхронная выборка через CompletableFuture (два источника объединяются в один объект
   async: запрошено 12, получено 11, сбоев 1, обращений к источникам 24, длительность 773.731
   потоки этапов: hw10-async-1, hw10-async-2, hw10-async-3, hw10-async-4, hw10-async-5, hw10-
   результаты (в порядке входного списка):
     v-01 | Backend-разработчик (Java) | Новосибирск | 70000–110000 ₽ | компания: АО «Сибирь-
     v-02 | Инженер по тестированию | Москва | 80000–130000 ₽ | компания: ООО «Код и Точка» (
     ...
     v-12 | Team Lead Java | Томск | 60000–110000 ₽ | компания: ООО «Код и Точка» (рейтинг 4.
     сбой v-07 (данные вакансии): SourceUnavailableException: источник /api/vacancies/v-07 от
   журнал «поток + ID запроса» (первые 14 строк):
     [hw10-async-1] v-01: запрос данных вакансии отправлен
     [hw10-async-2] v-01: запрос компании отправлен
     [hw10-async-3] v-02: запрос данных вакансии отправлен
     [hw10-async-4] v-02: запрос компании отправлен
     ...
```

Каждый идентификатор даёт **два** одновременных запроса (24 обращения на 12 идентификаторов), в результате участвовали все 6 потоков пула. Журнал показывает и имя потока, и ID запроса — по нему видно, что этапы разных вакансий действительно идут одновременно.

Этап 2. Синхронная версия и сравнение времени

Тот же сценарий выполнен последовательно (без пула) с той же семантикой изоляции сбоев:

```
Этап 2. Синхронная версия и сравнение времени
    sync: запрошено 12, получено 11, сбоев 1, обращений к источникам 23, длительность 2740.577
    синхронно 2740.577 мс, асинхронно 773.731 мс — ускорение 3.54× (в синхронной версии запрос
    на один идентификатор: синхронно 228.38 мс, асинхронно 64.48 мс
    наборы результатов совпадают: да (асинхронно 11, синхронно 11)
    обращений к источникам: асинхронно 24, синхронно 23
```

Ускорение **3.54×** при пуле 6 и 24 запросах. Теоретическая оценка: 24 × 120 мс / 6 потоков ≈ 480 мс плюс 12 × 10 мс преобразования ≈ 20 мс; практически 773 мс — разница приходится на запуск задач, HTTP-клиент и преобразование, которое тоже идёт на пуле. Асинхронная версия делает на одно обращение больше (24 против 23): синхронная после сбоя данных вакансии не запрашивает компанию, а конвейер запускает оба запроса сразу — это цена параллельности, а не ошибка.

Этап 3. Сбой одного источника: `handle` и `exceptionally`

```
Этап 3. Сбой одного источника не ломает операцию (handle и exceptionally)
    запрошено 3: [v-07, v-04, v-01]
    async: запрошено 3, получено 2, сбоев 1, обращений к источникам 6, длительность 176.151 мс
      v-07: исключён из результата: сбой источника вакансий обработан handle
      v-04: вакансия сохранена, компания подставлена заглушкой (сбой справочника обработан exc
      v-01: получен полностью, компания АО «Сибирь-Софт»
    вывод: сбой одной задачи не отменил остальные — метод вернул частичный результат
```

Два обработчика решают разные задачи:

- **`exceptionally`** стоит на будущем справочника компаний: сбой заменяется заглушкой `CompanyInfo.unknown(id)`, поэтому вакансия остаётся в результате с пометкой «компания: нет данных»;
- **`handle`** стоит на объединённом результате: если данных вакансии нет, идентификатор исключается, а сбой попадает в список `FetchFailure`; остальные конвейеры продолжают работать.

`handle` / `exceptionally` возвращают значение и не «пробрасывают» исключение дальше, поэтому `allOf(...).join()` завершается нормально, а метод возвращает частичный результат, а не падает.

3. Фактический вывод демонстрации

```
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw10 --server.port=0"
```

Прогон завершился с кодом `0`; живых потоков ДЗ 10 после остановки контекста нет.

```
=== ДЗ 10. CompletableFuture: асинхронная выборка по списку идентификаторов ===


Процесс: pid 128371
Настройки ДЗ 10:
```

```
    - идентификаторов в списке: 12
    - размер собственного пула: 6
    - задержка ответа источника: 120 мс на запрос
    - моделируемое тяжёлое преобразование: 10 мс на объект
    - таймаут запроса: 2000 мс, остановки пула: 3000 мс
    - сбой источника вакансий на «v-07», сбой справочника компаний на «v-04»
    - источник данных: локальная HTTP-заглушка, порт 41891

Этап 1. Асинхронная выборка через CompletableFuture (два источника объединяются в один объект)
    async: запрошено 12, получено 11, сбоев 1, обращений к источникам 24, длительность 773.731
    потоки этапов: hw10-async-1, hw10-async-2, hw10-async-3, hw10-async-4, hw10-async-5, hw10-
    ...

Этап 2. Синхронная версия и сравнение времени
    sync: запрошено 12, получено 11, сбоев 1, обращений к источникам 23, длительность 2740.577
    синхронно 2740.577 мс, асинхронно 773.731 мс — ускорение 3.54× (в синхронной версии запрос
    на один идентификатор: синхронно 228.38 мс, асинхронно 64.48 мс
    наборы результатов совпадают: да (асинхронно 11, синхронно 11)
    обращений к источникам: асинхронно 24, синхронно 23

Этап 3. Сбой одного источника не ломает операцию (handle и exceptionally)
    запрошено 3: [v-07, v-04, v-01]
    async: запрошено 3, получено 2, сбоев 1, обращений к источникам 6, длительность 176.151 мс
      v-07: исключён из результата: сбой источника вакансий обработан handle
      v-04: вакансия сохранена, компания подставлена заглушкой (сбой справочника обработан exc
      v-01: получен полностью, компания АО «Сибирь-Софт»
    вывод: сбой одной задачи не отменил остальные — метод вернул частичный результат

[tpm] Демонстрация завершена: частичный результат получен, сбой одного источника изолирован, и

после остановки контекста: пул остановлен=да, потоки завершены=да
живых потоков ДЗ 10 после завершения: []
```

Полный список результатов этапа 1 (все 11 вакансий с оценками) — в фактическом выводе прогона; выше он сокращён, чтобы отчёт не дублировал консоль целиком. Служебные строки Spring при остановке контекста (`GracefulShutdown`, закрытие JPA/Hikari) опущены — к демонстрации они не относятся.

4. Промежуточные результаты по этапам

Этап	Фактический результат
1. Асинхронная выборка	12 идентификаторов, 24 обращения к источникам, 11 результатов и 1 изолированный сбой, 6 потоков пула, **773.7 мс**
2. Синхронная версия	2740.6 мс, 1 поток, тот же набор результатов → **ускорение 3.54×**
3. Сбой одного источника	`v-07` исключён (`handle`), `v-04` сохранён с заглушкой компании (`exceptionally`), остальные получены

Этап	Фактический результат
Остановка	`пул остановлен=да`, `потоки завершены=да`, живых потоков `[]`

5. Почему не `commonPool` и чем `thenCombine` отличается от `thenCompose`

- **Собственный исполнитель.** `supplyAsync` без исполнителя и `thenApplyAsync` без исполнителя идут в `ForkJoinPool.commonPool()`, который делится со всеми (`parallelStream`, другие библиотеки). На нём нельзя ни задать размер, ни изолировать нагрузку, а его потоки называются `ForkJoinPool.commonPool-worker-N` — в дампе не видно, чьи это задачи. Поэтому все этапы получают явный `ExecutorService` с именами `hw10-async-N`.
- **Демон-потоки.** Пул создан фабрикой, которая помечает потоки демонами: аварийное завершение приложения не будет держать процесс. При этом пул всё равно закрывается явно (`destroyMethod = "close"`), потому что «процесс завершится сам» — не то же самое, что «ресурсы освобождены».
- **`thenApply` vs `thenApplyAsync(…, executor)`.** `thenApply` выполняется в потоке, который завершил предыдущий этап; для тяжёлого преобразования это означало бы, что поток, доставивший HTTP-ответ, занят CPU-работой и не берёт следующую задачу. `thenApplyAsync` с явным пулом возвращает поток в оборот.
- **`thenCombine` vs `thenCompose`.** `thenCombine` нужен, когда два будущих **независимы** и их результаты складываются (данные вакансии + компания). `thenCompose` был бы нужен, если бы второй запрос зависел от результата первого — здесь такой зависимости нет, и последовательное выполнение только удлинило бы операцию.
- **`thenAccept`** — терминальный этап без возврата значения: приём результата и запись в журнал. Он не меняет тип конвейера, поэтому после него `allOf` ждёт именно завершения обработки, а не только получения значения.

6. Автоматические проверки

```
mvn -pl app test -Dtest='ru.tsu.tpm.vacancyparser.hw10.**'
```

Результат: `Tests run: 30, Failures: 0, Errors: 0, Skipped: 0`.

Тест	Что проверяет
`AsyncVacancyServiceTest` (10)	порядок результатов; асинхронная версия быстрее синхронной и использует свой пул (имена `hw10-async-*`, ни одного `commonPool`); сбой вакансии исключает только свою запись; сбой компании даёт заглушку и не выбрасывает вакансию; полный сбой даёт пустой список без исключения; пустой список; потоки — демоны; журнал содержит имя потока и ID; `close()` останавливает пул; некорректные параметры
`VacancyStubServerTest` (6)	оба маршрута и разбор ответа; ошибки 500/503 с кодом; задержка применяется; порт освобождается; список идентификаторов; отрицательная задержка

Тест	Что проверяет
`FlatJsonTest` (4)	строки/числа/логические/null, экранирование, ошибки разбора
`AsyncFetchReportTest` (4)	ускорение и строка сравнения; пометки `handle` / `exceptionally`; строки результатов; обрезка журнала
`Hw10PropertiesBindingTest` (2)	значения по умолчанию и переопределение `app.hw10.*`
`CompletableFutureDemoConditionTest` (4)	демо и бины создаются только при `--app.hw=hw10`; после закрытия контекста пул остановлен и потоков нет

Сборка целиком: `mvn -pl app package` — успешно, `app/target/vacancy-parser.jar`. Полный прогон всех домашних заданий: `Tests run: 545, Failures: 0, Errors: 0, Skipped: 0`.

7. Соответствие чек-листу

Пункт задания	Где подтверждено	Отметка
Получение данных по списку идентификаторов из внешнего источника	Раздел 2, этап 1; `AsyncVacancyService`	✅
Асинхронно через `CompletableFuture` по каждому ID	`supplyAsync` на каждом источнике	✅
Преобразование каждого результата	`thenApplyAsync` + `VacancyScoring`	✅
Объединение двух независимых результатов в один объект	`thenCombine` → `FinalResult`	✅
Список итоговых объектов, собранных параллельно	`allOf` + порядок входного списка	✅
Использованы `thenCombine`, `thenApply`, `thenAccept`	`AsyncVacancyService#runAsync`	✅
`public List<FinalResult> fetchAllDataAsync(List<String> ids)`	`AsyncVacancyService`	✅
`handle` / `exceptionally`, сбой не ломает операцию	Раздел 2, этап 3; тесты изоляции	✅
Собственный `Executor`, не `commonPool`	`Executors.newFixedThreadPool` + фабрика имён	✅
Именованные демон-потоки, разумный размер	`hw10-async-N`, daemon=true, 6 потоков	✅
`thenApplyAsync(…, executor)` для тяжёлых преобразований	`AsyncVacancyService#runAsync`	✅
Корректное завершение при остановке приложения	`@Bean(destroyMethod = "close")`; проверка в тесте	✅
Логирование с именем потока и ID запроса	журнал `[hw10-async-3] v-07: …`	✅

Пункт задания	Где подтверждено	Отметка
Имитация ошибки в одном источнике → частичный результат	`v-07` (500), `v-04` (503)	✅
Сравнение времени async и sync	773.7 мс против 2740.6 мс (3.54×)	✅
Отчёт с этапами и промежуточными результатами	Этот файл	✅

8. Как запустить и сдать

```
cd "Теория и практика многопоточности"

# Полная демонстрация (завершается с кодом 0)
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw10 --server.port=0"

# Ускоренный прогон: маленький пул и быстрый источник
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw10 --app.hw10.pool-size=3

# Готовый jar
mvn -pl app package
java -jar app/target/vacancy-parser.jar --app.hw=hw10 --server.port=0
```

Все параметры выведены в начале прогона и меняются аргументами: `app.hw10.pool-size`, `app.hw10.latency-millis`, `app.hw10.transform-delay-millis`, `app.hw10.request-timeout-millis`, `app.hw10.ids`, `app.hw10.details-fail-id`, `app.hw10.company-fail-id`.

Сдача: скопировать содержимое этого файла в Google/Яндекс Документ, вложить исходники или ссылку на репозиторий, открыть доступ по ссылке и прикрепить ссылку в личном кабинете LMS.

9. Примечание о числах и окружении

Все числа — фактические замеры на этой машине (12 ядер, OpenJDK 21.0.13, Maven 3.9.16). Источник данных — локальная HTTP-заглушка на свободном порту: сеть не нужна, задержка ответа задаётся настройкой, поэтому прогон воспроизводим. Времена между запусками различаются на единицы процентов (в первую очередь из-за запуска JVM и планировщика), структура вывода устойчива, а абсолютное ускорение зависит от размера пула и задержки источника.